

UNIVERSIDAD TECNOLÓGICA NACIONAL

Carrera: Tecnicatura Universitaria en Programación

1er cuatrimestre 2026

Materia: Programación III

Profesor: Gustavo Ramoscelli

Ayudante: Maria Victoria Ruiz

Trabajo Práctico N°3: Node.js, Express, npm, consumo de APIs y deploys

Objetivos:

- ❖ Aplicar los conocimientos vistos en clase sobre modularización, Node.js, express y otros módulos
- ❖ Instalación de paquetes por medio de la terminal con 'npm'.
- ❖ Aplicar conocimientos iniciales de POO (Programación Orientada a Objetos).
- ❖ Gestionar datos mediante archivos JSON para simular una base de datos.
- ❖ Diseñar y desarrollar endpoints para una API REST, utilizando los métodos 'GET' y 'POST'.
- ❖ Utilizar las herramientas de 'render.com' para hacer un deploy de una API.
- ❖ Trabajar en equipo utilizando herramientas de Git y GitHub.

Fecha de entrega:

- ❖ **Miércoles 13 de mayo 23:59hs.** Se tomará como entrega el último commit y merge en la rama main realizado antes de la fecha límite. ***Por excepción, está permitido realizar hacer varios pull request a 'main' para experimentar con 'Render', pero deben seguir realizando los commits en las ramas correspondientes.***

Enunciado:

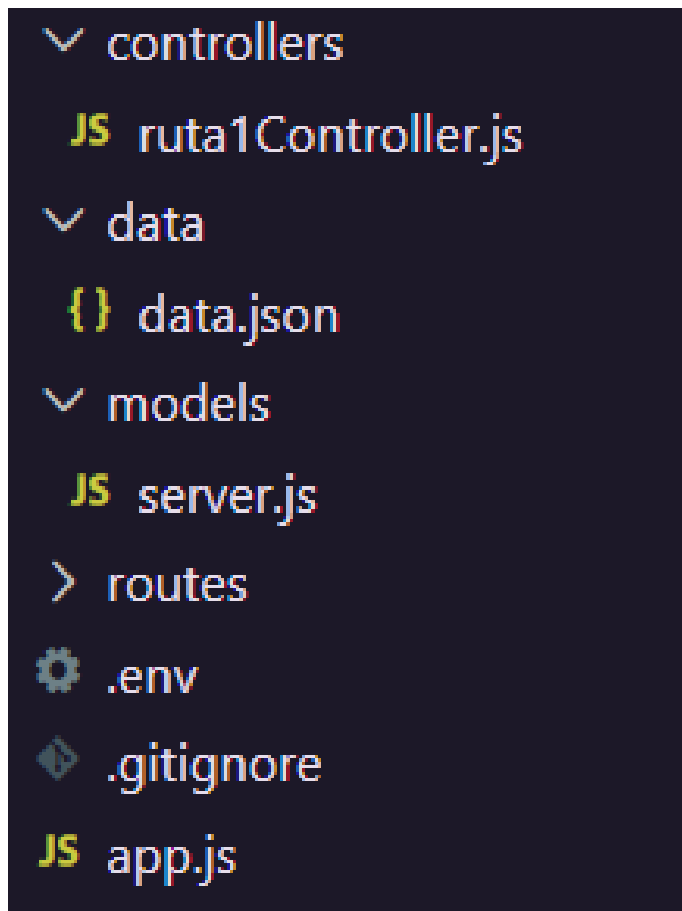
El objetivo es crear una API REST para reemplazar los datos estáticos de una página y, en su caso, empezar a consumir una API.

Se deberá modificar el código del proyecto realizado para el TP n°1 (pueden realizar un fork) para obtener los datos del back-end desde el deploy que se realiza desde 'Render'.

Es obligatorio documentar correctamente bajo la misma estructura que el examen n°1, está detallado en el último ítem de 'Requisitos Técnicos'.

Se deberá hacer el back-end para los apartados de servicios y equipo; sumado a esto, se deben agregar dos páginas y apartados nuevos para un login de usuarios y un apartado del perfil de usuario, pueden copiar y pegar el código del 'TP2 - Cine Aurora' y modificar lo necesario para que sea acorde al proyecto que eligieron.

Archivos iniciales (sin instalar módulos):



❖ **app.js**: Es nuestro archivo inicial y lo que Node.js va a

❖ **.gitignore**: Con este archivo le indicamos a Git y GitHub qué no queremos que se suba al repositorio. En el ejemplo de la clase del día '04/05/26' tienen una plantilla de lo que deben agregar en general.

❖ **.env**: Este es un archivo que guarda variables que no deben ser públicas pero vamos a consultar; por todo esto va a ser un archivo exclusivamente local. **Es obligatorio agregarlo al archivo '.gitignore'.**

❖ **./models**: En esta carpeta vamos a tener un archivo por clase. Para poder empezar el servidor de la API siempre vamos a tener el archivo 'server.js'.

❖ **./data**: En esta carpeta vamos a tener los archivos JSON que se van a consultar como si fuera una base de datos.

❖ **./controllers**: **OPCIONAL**: No es obligatorio, pero por cada ruta nueva que le agregamos al proyecto, se puede separar la lógica en archivos diferentes, y estos se deben guardar acá. Es una buena práctica agregar 'Controller' al final del nombre del archivo, por ejemplo: 'serviciosController'.

Acciones a realizar:

1. Revisar que tenemos instalado Node.js y npm ('node -v' y 'npm -v' desde una terminal).
2. Instalar y configurar la extensión de VS Code 'ESLint' de Microsoft, contiene más de 50 millones de descargas.
3. Inicializar repositorio con Git o clonar desde GitHub.
4. Crear las carpetas y archivos necesarios establecidos en la imagen, agregando los de configuración explicados en clase.
5. Abrir una terminal en la carpeta correcta, e inicializar un proyecto de Node.js (npm init). Se deben completar varios campos desde la terminal.

6. Instalar todos los módulos necesarios de npm (express, nodemon, cors, .env, husky, etc).
 7. Revisar que se nos haya creado la carpeta './node_modules'; en el caso de que no esté presente realizamos mal la instalación.
 8. Revisar que estén los módulos instalados en el nuevo archivo 'package.json' dentro del apartado 'dependencies'.
 9. Configurar dentro del apartado 'scripts' en 'package.json'.
 10. Modificar el trabajo práctico n°1 para realizar un deploy con GitHub Pages y cambiar las funciones existentes de JS para que ahora sean asíncronas y consuman nuestra API.
 11. Crear una cuenta en 'Render' y configurar nuestro deploy a partir de nuestro repositorio del TP n°1.
 12. Happy coding!
-

Endpoints requeridos (rutas)

Deberán

- ❖ GET/servicios: Retorna el listado de servicios ofrecidos. Es obligatorio que haya más de 13 servicios.
 - ❖ GET/servicios/:id: Retorna el detalle de un servicio en particular.
 - ❖ GET/equipo: Retorna los integrantes del equipo.
 - ❖ GET/perfil/:id: Retorna los datos del usuario logueado (nombre, mail, fecha de registro, foto, últimos 3 pedidos).
-

Requisitos Técnicos

El proyecto debe cumplir estrictamente con las siguientes pautas de desarrollo:

- ❖ Deploy en render: Es obligatorio que la API (back-end) esté subida a 'Render.com' y responda correctamente a las peticiones externas.
- ❖ Utilización del mismo repositorio del TP n°1: Se debe trabajar sobre el mismo repositorio existente, manteniendo la coherencia entre el front-end y el nuevo back-end. Es obligatorio modificar el proyecto ya que se deben consumir los datos directamente desde la URL de la API en Render, no desde archivos locales.
- ❖ Deploy en gh-pages: Es obligatorio tener un deploy del front-end en GitHub Pages.

- ❖ **Cumplir con la división de carpetas y nombres correctos:** Se debe organizar el código respetando las carpetas y archivos indicados en el enunciado.
 - ❖ **Utilizar funciones asíncronas con bloques try/catch:** Es obligatorio estructurar el manejo de la lectura de archivos JSON y las respuestas de las rutas de forma asíncrona, asegurando que el servidor no se caiga ante errores inesperados.
 - ❖ **Commits y pull requests:** Para probar las funcionalidades de la herramienta 'Render', está permitido en este trabajo práctico realizar varios pull request desde 'dev' a 'main'.
 - ❖ **Documentación:** El archivo README.md debe incluir lo siguiente:
 - Número de grupo e integrantes,
 - Nombre del proyecto y su descripción.
 - Metodología de trabajo con Git y GitHub.
 - División de los archivos entre los integrantes.
 - Distribución de los archivos y carpetas.
 - Un 90% de las funciones explicadas a detalle.
 - Mínimo un ejemplo de la estructura de cada archivo JSON utilizado (no integrar varios "arrays" en un mismo archivo).
-

Opcional

Pueden agregar lo siguiente:

- ❖ **POST/login:** Crear un nuevo endpoint que recibe desde el front-end un usuario y contraseña, y chequea si los datos son correctos, simulando con un JSON la información de los usuarios registrados.
 - ❖ **Establecer 'flags':** Es una buena práctica mostrar por la consola qué se ejecutó por medio de un mensaje. Utilizar 'console.log()' para los comentarios.
 - ❖ **Efectos de Carga:** Implementar un "Spinner" o mensaje de "Cargando..." en el front-end mientras se espera con las promesas la respuesta de la API de Render.
 - ❖ **Registrar usuarios:** Registrar un nuevo usuario modificando y escribiendo el JSON correspondiente con los datos de los usuarios.
 - ❖ **Buscador Sencillo:** Lograr que filtren los servicios del JSON por su nombre, puede ser con un endpoint o desde el front-end.
-

Entrega

El trabajo deberá entregarse mediante un repositorio en GitHub, la fecha límite es el **miércoles 13 de mayo 23:59hs.**

El último commit y merge de la rama main es el que se toma como entrega del trabajo práctico. ***Por excepción, se permite hacer varios pull request desde 'dev' a 'main' para experimentar con 'Render', pero deben seguir realizando los commits en las ramas correspondientes.***

La metodología de trabajo con Git y GitHub tiene que ser en ramas. Se puede utilizar git flow (por features) o una rama por alumno. Aclarar esto en la documentación del archivo 'README.md'.

Obligatoriamente, cada alumno debe tener como mínimo un commit en su rama y un pull request para cada merge.